S01 Foundations

Malware Research and Linux Practical Work

Raja Wardet Wahaj

Cohort2: DFCS Bootcamp — Atomcamp

Repository path: https://github.com/WardetWahaj/<your-repo>/tree/main/s01-foundations

Tasks completed: Task A, B2, B3

Environment: Kali Linux / Linux terminal; Python 3

Task A — Malware Types and Real-World Examples

The word malware describes software or code used to perform unwanted or harmful actions. The categories below describe different properties: some describe how code spreads (for example, a worm), while others describe its purpose or concealment method (for example, spyware or a rootkit). A single malware family can therefore fit more than one category.

Type	Definition	Example + year	How it spread	Primary impact
Virus	Inserts its code into a host file, document, or boot area; normally needs that host to be run or accessed before it can infect further items.	Brain (1986) [1][2]	Carried on infected 5.25-inch floppy disks; when used on an IBM PC-compatible, Brain copied itself to other floppy disks accessed by that machine.	Replaced the floppy disk's boot sector and moved the original elsewhere. Written by two brothers in Lahore, Pakistan; designed to replicate and identify itself, not to destroy files.
Worm	Makes and distributes copies of itself to other systems without needing to attach to a separate host program.	Morris worm (1988) [3][4]	Spread among networked Unix systems on the early Internet by exploiting weaknesses in sendmail and finger, and by guessing weak passwords.	Repeated self-replication consumed resources and disrupted thousands of networked computers; the FBI calls it the first major attack on the Internet.
Trojan	Presents itself as useful or legitimate software but performs hidden malicious actions after execution. Does not self-replicate.	Zeus / Zbot (2007) [5][6]	Distributed through phishing and drive-by downloads exploiting vulnerable browsers or applications.	Stole online-banking credentials, enabling account access and fraudulent transfers; also provided a platform for building botnets.
Ransomware	Denies access to data or systems, usually by encrypting files, and demands payment for restoring access.	WannaCry (2017) [7][8]	Exploited the Windows SMB vulnerability (EternalBlue) to spread between reachable, unpatched computers.	Encrypted commonly used files and displayed a ransom demand; healthcare and other organisations worldwide experienced disruption.
Spyware	Covertly monitors a person's device or activity and sends collected information to an operator.	Pegasus (2016 targeting documented) [9][10]	2016 case: SMS links to NSO exploit infrastructure triggered an install; later campaigns also used zero-click techniques.	Enabled intrusive surveillance: messages, calls, location, microphone and camera. Citizen Lab documented targeting of civil-society figures.
Adware	Displays or injects advertising; malicious adware hijacks browser settings or redirects searches without meaningful consent.	Fireball (2017) [11]	Commonly bundled with freeware and browser-related downloads users wanted to install.	Hijacked browsers and manipulated web traffic for ad revenue; capabilities also allowed code execution and

Type	Definition	Example + year	How it spread	Primary impact
				downloading further malware.
Rootkit	Changes or abuses low-level system components so an attacker or unwanted program can hide files, processes or other evidence of its presence.	Sony BMG XCP rootkit (2005) [12][13]	Installed on Windows computers when users played certain Sony BMG music CDs and accepted the install prompt.	Concealed files and registry objects tied to copy-protection software, creating security/privacy risk and a hiding place for other malicious activity.
Keylogger	Records keys typed by a user; may be standalone or a feature inside a larger info-stealing Trojan.	Agent Tesla (active since 2014) [14][15]	Commonly delivered in phishing emails with malicious Office-document attachments disguised as business correspondence.	Records keystrokes and can steal saved credentials, clipboard contents and screenshots, exposing email, VPN and browser accounts.
Botnet agent	Turns an infected device into a remotely controlled member of a larger group of compromised devices.	GameOver Zeus (2011) [16]	Infected machines via the Zeus malware ecosystem (phishing, malicious downloads); used a decentralised peer-to-peer C2 design.	Silently collected banking credentials; DOJ estimated 500,000–1,000,000 infected computers and over US$100 million in losses.
Fileless / LOTL	Executes or persists without relying on a conventional malicious executable saved to disk; abuses legitimate tools already present (e.g. PowerShell, WMI).	Documented by Microsoft (2018) [17][18]	A backdoor was deployed and a subtler channel used registry-stored, obfuscated PowerShell code; initial access vector varied.	Payload ran without dropping a conventional executable and used a registry Run key for persistence, reducing file-based scanning effectiveness.
Logic bomb	Code planted in a system that stays inactive until a specified condition occurs, such as a date or account change.	UBS PaineWebber logic bomb (2002) [19]	A privileged systems administrator allegedly planted the code before leaving the company; it did not self-spread.	Activated on 4 March 2002 and deleted files on 1,000+ computers; DOJ said damage assessment and repair cost exceeded US$3 million.
Bootkit	Infects the boot process (e.g. the MBR) so malicious code runs before the OS has fully started.	TDL-4 (2011) [20][21]	Installed by the TDSS/TDL Trojan family via varied delivery methods; the bootkit component modified the MBR after reaching the machine.	Modified the MBR to gain control before the OS loader, making detection and removal harder for tools that operate only after the OS starts.

Which type is most dangerous for a Pakistani SME?

For a typical Pakistani small or medium-sized business, ransomware is the most dangerous single malware type because one successful incident can make accounting records, customer files, invoices, and shared business data unavailable at the same time. SMEs typically have limited IT staff, weak patch management, and backups connected to the same network, so recovery is expensive and prolonged. WannaCry demonstrates how a single unpatched SMB-exposed computer can escalate into a wider incident. A practical defense is to patch internet-facing and networked systems promptly, restrict unnecessary SMB exposure, use tested offline or immutable backups, enable endpoint protection, and train staff to recognize phishing.

Task B2 — Linux File Permissions

Objective

Create a text file and demonstrate three permission changes using both symbolic and octal chmod notation. ls -l shows the human-readable permission string, while stat prints the numeric octal mode alongside it.

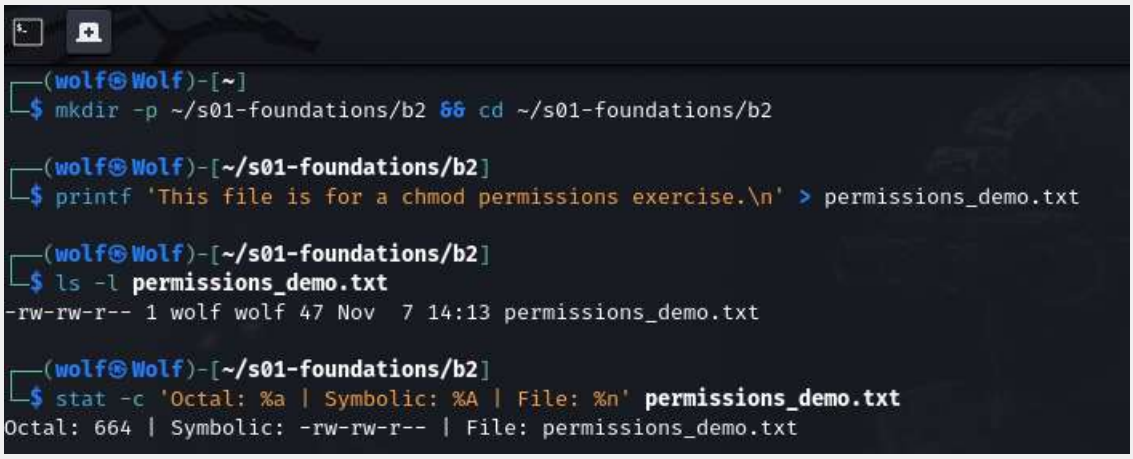
Permission notation

Symbol	Meaning
u	the file owner (user)
g	users in the file's group
o	everyone else (other)
r	read (value 4)
w	write (value 2)
x	execute (value 1)

For an ordinary file, the three octal digits represent owner, group, and other permissions. The values in each digit are added: rwx = 4+2+1 = 7, rw- = 4+2 = 6, r-- = 4, --- = 0.

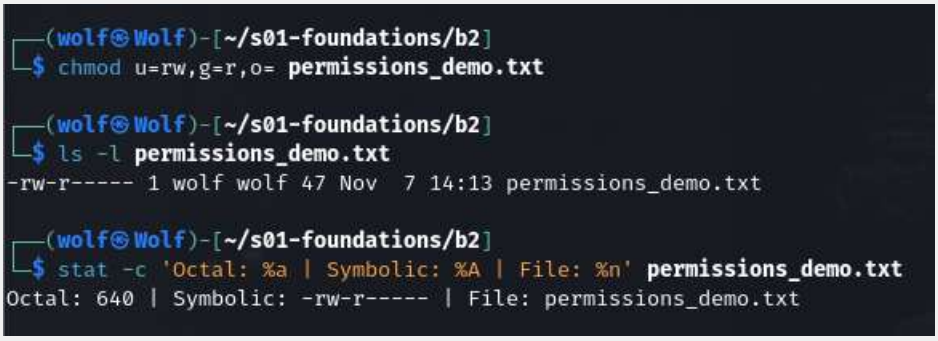
Setup and initial state

```
mkdir -p ~/s01-foundations/b2 && cd ~/s01-foundations/b2
printf 'This file is for a chmod permissions exercise.\n' > permissions_demo.txt
ls -l permissions_demo.txt
stat -c 'Octal: %a | Symbolic: %A | File: %n' permissions_demo.txt
```



Change 1 — symbolic assignment (u=, g=, o=)

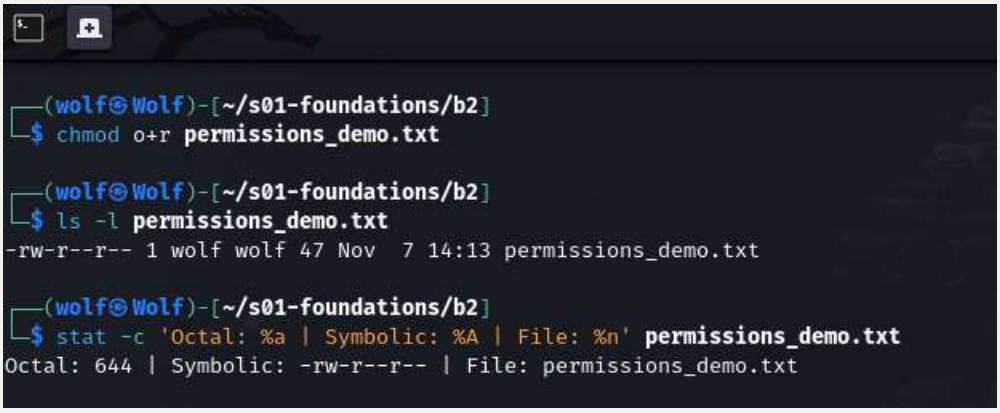
```
chmod u=rw,g=r,o= permissions_demo.txt
ls -l permissions_demo.txt
stat -c 'Octal: %a | Symbolic: %A | File: %n' permissions_demo.txt
```



Explanation: the owner can read and write; the group can read; other users receive no permissions. The = operator sets the selected permission bits rather than merely adding or removing one permission.

Change 2 — symbolic addition (+)

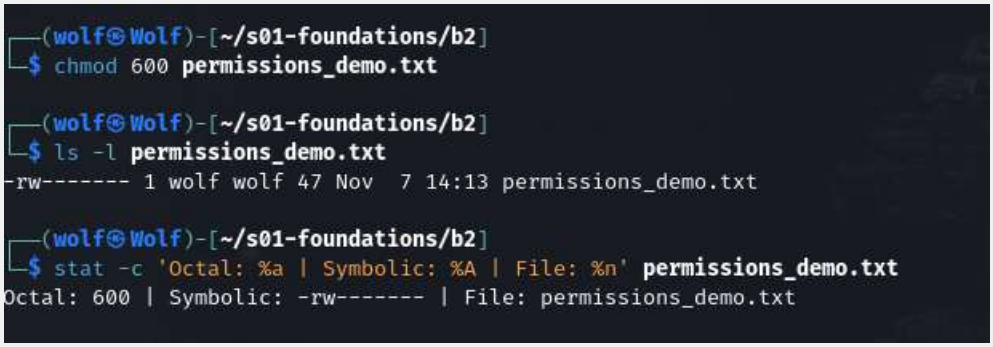
```
chmod o+r permissions_demo.txt
ls -l permissions_demo.txt
stat -c 'Octal: %a | Symbolic: %A | File: %n' permissions_demo.txt
```



o+r adds read permission for other users while retaining the existing owner and group permissions.

Change 3 — octal assignment

```
chmod 600 permissions_demo.txt
ls -l permissions_demo.txt
stat -c 'Octal: %a | Symbolic: %A | File: %n' permissions_demo.txt
```



6 for the owner means read+write (4+2); 0 for the group and 0 for others means no permissions for either. This is a common restrictive setting for a private file.

B2 results summary

Stage	Command / mode	Expected ls -l	Meaning
Initial	Created under current umask	Depends on VM — record actual output	Baseline before any chmod
Change 1	chmod u=rw,g=r,o= — 640	-rw-r-----	Owner read/write; group read; others none
Change 2	chmod o+r — 644	-rw-r--r--	Adds read permission for others
Change 3	chmod 600 — 600	-rw-------	Owner read/write only

Replace the "Expected" column with your actual terminal output and embed a screenshot after each command block before submitting.

Task B3 — Caesar Cipher Decoder in Python

Objective

Write a Python program that accepts a Caesar-shifted string and prints all 25 non-zero possible shifts, preserving spaces, punctuation and letter case.

Script: caesar_decoder.py

```
#!/usr/bin/env python3
import argparse
import string

def caesar_decode(text: str, shift: int) -> str:
    """Decode text by shifting letters backwards; preserve case and punctuation."""
    result = []
    for char in text:
        if char in string.ascii_lowercase:
            index = (ord(char) - ord("a") - shift) % 26
            result.append(chr(ord("a") + index))
        elif char in string.ascii_uppercase:
            index = (ord(char) - ord("A") - shift) % 26
            result.append(chr(ord("A") + index))
        else:
            result.append(char)
    return "".join(result)

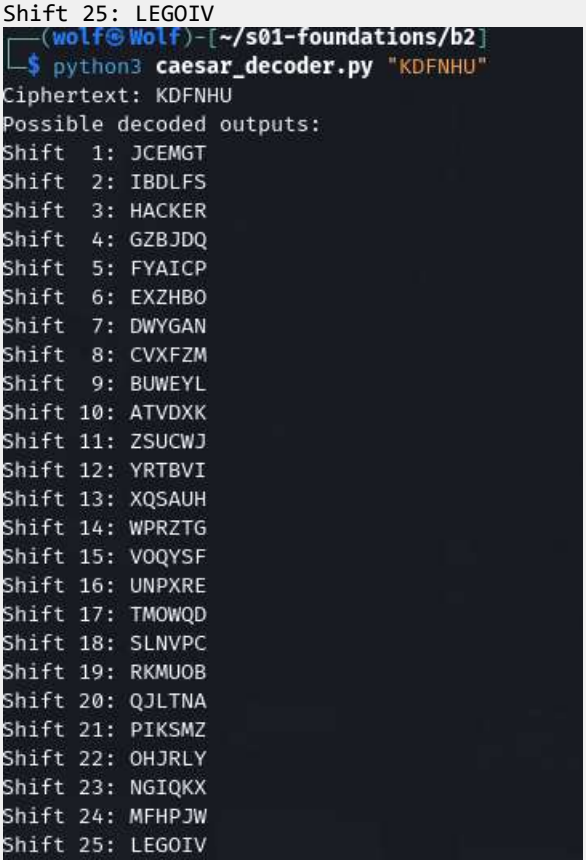
def main() -> None:
    parser = argparse.ArgumentParser(description="Try all 25 non-zero Caesar cipher shifts.")
    parser.add_argument("text", help='The shifted text, for example: "Khoor, Zruog!"')
    args = parser.parse_args()
    print(f"Ciphertext: {args.text}")
    print("Possible decoded outputs:")
    for shift in range(1, 26):
        print(f"Shift {shift:2d}: {caesar_decode(args.text, shift)}")

if __name__ == "__main__":
    main()
```



Example run

```
python3 caesar_decoder.py "KDFNHU"
Ciphertext: KDFNHU
Possible decoded outputs:
Shift  1: JCEMGT
Shift  2: IBDLFS
Shift  3: HACKER      <-- readable, key = 3
Shift  4: GZBJDQ
...
Shift 25: LEGOIV
```



Why 25 tries is not security

A Caesar cipher has only 25 meaningful non-zero shifts for English letters, so a computer can test every possible key almost instantly. It also preserves letter-frequency patterns, making it vulnerable to simple frequency analysis. It is useful for learning substitution ciphers, but it must not be used to protect real passwords, files, or communications.

GitHub Repository Structure

```
s01-foundations/
├── README.md
├── caesar_decoder.py
└── figures/
    ├── b2-1-initial.png
    ├── b2-2-symbolic-640.png
    ├── b2-3-symbolic-644.png
    ├── b2-4-octal-600.png
    └── b3-caesar-run.png
```

References

1. Computing History, "Brain, the first PC virus, is released" (19 January 1986). https://www.computinghistory.org.uk/det/6303/

2. IBM, "The History of Malware". https://www.ibm.com/think/topics/malware-history

3. FBI, "Morris Worm", Major Cases. https://www.fbi.gov/investigate/cyber/major-cases

4. IBM, "The History of Malware" (Morris worm propagation and impact). https://www.ibm.com/think/topics/malware-history

5. IBM X-Force, 2011 Trend and Risk Report. https://www.fbiic.gov/public/2012/mar/2011IBMReport-March2012.pdf

6. NHS England Digital, "Zeus Trojan". https://digital.nhs.uk/cyber-alerts/2017/cc-1138

7. NHS England Digital, "WannaCry Ransomware Using SMB Vulnerability" (2017). https://digital.nhs.uk/cyber-alerts/2017/cc-1411

8. CERT-EU, "WannaCry Ransomware Campaign Exploiting SMB Vulnerability" (2017-012). https://cert.europa.eu/publications/security-advisories/2017-012/

9. Citizen Lab, "The Million Dollar Dissident" (2016). https://citizenlab.ca/research/million-dollar-dissident-iphone-zero-day-nso-group-uae/

10. Amnesty International Security Lab, "The Pegasus Project". https://securitylab.amnesty.org/case-study-the-pegasus-project/

11. Check Point Research, "Fireball" (1 June 2017). https://blog.checkpoint.com/research/fireball-chinese-malware-250-million-infection/

12. Electronic Frontier Foundation, Sony BMG Litigation. https://w2.eff.org/IP/DRM/Sony-BMG/

13. Mark Russinovich, "Inside Sony's rootkit", Virus Bulletin (December 2005). https://www.virusbulletin.com/virusbulletin/2005/12/inside-sony-s-rootkit/

14. Fortinet FortiGuard Labs, Agent Tesla phishing campaign (10 December 2021). https://www.fortinet.com/blog/threat-research/phishing-campaign-targeting-korean-to-deliver-agent-tesla-new-variant

15. Netskope Threat Labs, Agent Tesla malware profile. https://threatlabs.netskope.com/library/malware/agent-tesla

16. US Department of Justice, GameOver Zeus / Cryptolocker action (2 June 2014). https://www.justice.gov/archives/opa/pr/us-leads-multi-national-action-against-gameover-zeus-botnet-and-cryptolocker-ransomware

17. Microsoft Security Blog, "Now you see me: Exposing fileless malware" (24 January 2018). https://www.microsoft.com/en-us/security/blog/2018/01/24/now-you-see-me-exposing-fileless-malware/

18. Microsoft Learn, "Fileless threats". https://learn.microsoft.com/en-us/defender-endpoint/malware/fileless-threats

19. US Department of Justice, UBS PaineWebber logic bomb case (17 December 2002). https://www.justice.gov/archive/criminal/cybercrime/press-releases/2002/duronioIndict.htm

20. ESET, "When I'm x64: Bootkit threat evolution" (TDL-4, 2011). https://www.eset.com/fileadmin/eset/US/resources/docs/white-papers/white-papers-when-im-x64-bootkit-threat-evolution-2011.pdf

21. Check Point, "Rootkit: TDLv4" (4 July 2011). https://advisories.checkpoint.com/defense/advisories/public/2011/cpai-2011-323.html

22. GNU Project, GNU Coreutils Manual — chmod invocation. https://www.gnu.org/software/coreutils/manual/html_node/chmod-invocation.html

23. GNU Project, GNU Coreutils Manual — File permissions. https://www.gnu.org/software/coreutils/manual/html_node/File-permissions.html